Star Schema Data Warehouse — Explained

What is a Star Schema?

A star schema is a data warehouse design pattern where one central fact table is surrounded by multiple dimension tables, connected through simple foreign key relationships. If you draw it out, the fact table sits in the middle with dimension tables radiating outward — like a star.

- Fact table → the numbers, the "what happened" (transactions, events, measurements)
- Dimension tables → the descriptive context, the "who, what, where, when"

The Simple Version (Pizza Shop Analogy)

Imagine you own a pizza shop and want to answer: "How much money did I make, broken down by customer and pizza type?" You keep three notebooks:

Notebook A — "Sales" (the FACT table)

Every sale gets one line, just numbers and ID codes:

```
Order #123 | Customer #5 | Pizza #9 | Date: today | Amount: $15
```

Notebook B — "Customers" (a DIMENSION table)

```
Customer #5 = Fulya, from Netherlands, born 1997
```

Notebook C — "Pizzas" (another DIMENSION table)

```
Pizza #9 = Margherita, category: Vegetarian, cost to make: $4
```

Why split it up instead of one giant notebook?

- **No repeating yourself —** if Fulya buys 100 pizzas, you don't rewrite her details 100 times. You just write "Customer #5" and look her up once.
- **Easy to fix mistakes —** if Fulya moves to Germany, you update it in ONE place (Notebook B), not in every sales row.
- **Easy questions —** "Total sales by country?" → check Notebook A, hop to Notebook B for country, done. Simple hop instead of digging through a messy giant table.

Other ways to picture it

Analogy	Fact table	Dimension tables
Wheel	Hub (center)	Spokes radiating out
Excel	"Sales" sheet with just numbers/IDs	"Customer Info" / "Product Info" sheets you VLOOKUP into
Restaurant receipt	The receipt (Item #4, Qty 2, $10)	The menu (full descriptions), referenced but not reprinted every time

The Technical Version (Applied to a Real Script)

Example: a Gold-layer SQL script (Medallion architecture) that builds a star schema from Silver-layer tables.

Dimension: gold.dim_customers

```
CREATE VIEW gold.dim_customers AS
SELECT
    ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key, -- Surrogate key
    ci.cst_id                          AS customer_id,
    ci.cst_key                         AS customer_number,
    ci.cst_firstname                   AS first_name,
    ci.cst_lastname                    AS last_name,
    la.cntry                           AS country,
    ci.cst_marital_status              AS marital_status,
    CASE
        WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr -- CRM is primary source
        ELSE COALESCE(ca.gen, 'n/a')               -- Fallback to ERP data
    END                                AS gender,
    ca.bdate                           AS birthdate,
    ci.cst_create_date                 AS create_date
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid;
```

Dimension: gold.dim_products

```
CREATE VIEW gold.dim_products AS
SELECT
    ROW_NUMBER() OVER (ORDER BY pn.prd_start_dt, pn.prd_key) AS product_key,
    pn.prd_id       AS product_id,
    pn.prd_key      AS product_number,
    pn.prd_nm       AS product_name,
    pn.cat_id       AS category_id,
    pc.cat          AS category,
    pc.subcat       AS subcategory,
    pc.maintenance  AS maintenance,
    pn.prd_cost     AS cost,
    pn.prd_line     AS product_line,
    pn.prd_start_dt AS start_date
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id
WHERE pn.prd_end_dt IS NULL; -- Filter out historical data, keep current only
```

Fact: gold.fact_sales

```
CREATE VIEW gold.fact_sales AS
SELECT
    sd.sls_ord_num  AS order_number,
    pr.product_key  AS product_key,
    cu.customer_key AS customer_key,
    sd.sls_order_dt AS order_date,
    sd.sls_ship_dt  AS shipping_date,
    sd.sls_due_dt   AS due_date,
    sd.sls_sales    AS sales_amount,
    sd.sls_quantity AS quantity,
    sd.sls_price    AS price
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products pr ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu ON sd.sls_cust_id = cu.customer_id;
```

Mapping analogy → real script

Pizza shop	Real script
Notebook A (Sales)	gold.fact_sales — the transactions

Notebook B (Customers)	gold.dim_customers — who bought stuff
Notebook C (Pizzas)	gold.dim_products — what got bought
Customer/Pizza ID numbers	customer_key / product_key — surrogate keys linking the tables

Key concepts explained

1. Surrogate keys

ROW_NUMBER() OVER (...) generates a clean, warehouse-owned integer ID (customer_key, product_key) — separate from the "natural" business key from the source system (cst_id, prd_id). Surrogate keys are stable and don't break if source-system ID logic changes.

2. Fact tables stay "skinny"

Notice fact_sales doesn't repeat the customer's name or product category — it only references keys (customer_key, product_key) and holds measures (sales_amount, quantity, price). Descriptive/textual attributes live in dimensions, not facts.

3. Data quality fallback logic

In dim_customers, gender uses CRM as the trusted source, falling back to ERP data via COALESCE if CRM says 'n/a'. This is a simple master-data-management pattern: deciding which source "wins" when systems disagree.

4. Current-state filtering

WHERE pn.prd_end_dt IS NULL in dim_products keeps only active product versions, excluding historical records. This makes it a "current state" dimension, not a full-history (Type 2 slowly changing dimension) table.

5. Views vs. tables

These are VIEWs, not physical tables — so the Gold layer is computed fresh from Silver at query time. Trade-off: always up to date and simple to maintain, but potentially slower than materialized/indexed tables on very large datasets.

Why star schema is the standard for BI/warehousing

- **Query simplicity —** simple fact → dimension joins, no deep join chains
- **Performance —** BI tools (Power BI, Tableau, etc.) are optimized to recognize and efficiently aggregate star schemas
- **Business-friendly —** "Total sales by country and category?" is a straightforward join: fact_sales → dim_customers → dim_products

Related Concepts to Explore Next

- **Snowflake schema —** the normalized cousin of star schema, where dimensions are further split into sub-tables (trades some query simplicity for less data redundancy)
- **Slowly Changing Dimensions (SCD) Type 2 —** how to track full history of changes in a dimension (e.g., product price changes over time) instead of only current state
- **Medallion architecture (Bronze/Silver/Gold) —** the broader layered pattern this script fits into: raw → cleaned → business-ready